

COMPASS_Synop- tic_LE_Tree_CO2_Flux_Calcs_2023b

Roberta Peixoto

2026-04-28

Table of contents

1	Add Required Packages	2
2	Pull in Raw Datafiles	2
3	Read in metadata file and match to raw data	4
4	Check start times overlaps or missing values	5
5	Match metadata to raw data	6
6	reating alldata matched df	8
7	Creating temperature csv from the L2-weather station in the wetland	9
8	Reading Temp df to merge to metadat	10
9	fixing some Deadbands and stop time	11
10	Calculate fluxes	12
11	check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)	20
12	Spot check a few fluxes with $R^2 > 0.9$	22
13	Take a look at the fluxes < 0.9 and a flux estimate below 1	27
14	Take a look at the bad fluxes < 0.9 and have a high flux estimate	30

15 See if we want to remove these fluxes or if we want to try and recalculate these ones?	30
16 Look at the really high or really low ones to see if there is a timing issue or something	33
17 Check all fluxes	35
18 QAQC - Flags	39
19 Adding manual flag to reject any fluxes that do not fit criteria	42
20 Do some visualization of QAQC plots to assess fluxes	43
21 Read out final dataframe and export for use elsewhere	46

1 Add Required Packages

2 Pull in Raw Datafiles

```
#defining where the raw data is

DATA_DIR_ROOT <- here::here(
  "Tree_Fluxes_Synoptic_LE",
  "2023",
  "Raw Data"
)

#function to read data
now <- function() format(Sys.time(), "%Y-%m-%d %H:%M:%S")
message(now(), " | Script started")
#finding the zip files
message(now(), " | Looking for ZIP files in: ", DATA_DIR_ROOT)

zip_files <- list.files(
  DATA_DIR_ROOT,
  pattern = "\\\\.zip$",
  full.names = TRUE)

message(now(), " | ZIP files found: ", length(zip_files))
```

```

print(zip_files)
  if (length(zip_files) == 0) {
    stop("No ZIP files found. Check DATA_DIR_ROOT.")}

cache <- list()

# Reader function
read_li7810_zip <- function(zip_file, tz = "US/Eastern") {
  message(now(), " | Processing ZIP: ", basename(zip_file))
  if (zip_file %in% names(cache)) {
    message(" -> Using cached version")
    return(cache[[zip_file]]) }

  tmp_dir <- tempfile("li7810_")
  dir.create(tmp_dir)
  on.exit(unlink(tmp_dir, recursive = TRUE), add = TRUE)
  unzip(zip_file, exdir = tmp_dir)
  data_files <- list.files(
    tmp_dir,
    pattern = "\\\\.data$",
    full.names = TRUE,
    recursive = TRUE )

  message(" -> .data files found: ", length(data_files))
  if (length(data_files) == 0) {
    stop("No .data files inside ZIP: ", zip_file) }

  dat_list <- lapply(data_files, function(f) {
    message("      Reading: ", basename(f))
    ffi_read_LI7810(f)})

  dat <- data.table::rbindlist(dat_list, fill = TRUE) %>%
    mutate(TIMESTAMP = with_tz(TIMESTAMP, tz))
  cache[[zip_file]] <- dat
  message(" -> ZIP finished: ", basename(zip_file))
  return(dat)}

# ACTUAL EXECUTION
message(now(), " | Starting data read")
alldat_list <- lapply(zip_files, read_li7810_zip)
alldat <- data.table::rbindlist(alldat_list, fill = TRUE)
message(now(), " | DONE")

```

```

message("Final number of rows: ", nrow(alldat))

# Show result
print(head(alldat))

#REMOVING WEIRD VERY LOW VALUES ON "2022-12-20 11:41:00" co2=-4000 licor bug???
alldat<-alldat%>%filter(CO2>-1000)

```

3 Read in metadata file and match to raw data

```

md <- "Raw Data/COMPASS_SynopticLE_TreeFlux_Metadata_2023.csv"
dat <- read.csv(md)

metadat <- dat %>%
  # Remove rows without date
  filter(!is.na(Date)) %>%
  # Keep only desired instruments
  filter(SN %in% c("TG10-01448", "TG10-01426", "TG10-01028")) %>%
  # Normalize Plot labels
  mutate(
    Plot_raw = toupper(Plot),
    # Extract plot components
    #Site = str_extract(Plot_raw, "[A-Z]{3}"),
    #Zone = str_extract(Plot_raw, "(TR|UP)"), # somehow all of the UP PTR measurements turned
    Site = toupper(Site),
    Zone = toupper(Zone),
    Replicate = str_extract(Plot_raw, "(?<=TR|UP)[0-9]+"),
    # Letter ONLY if it comes AFTER a number
    Letter = str_extract(Plot_raw, "(?<=[0-9])[A-Z]"),
    # Time normalization
    Time_start = str_replace_all(as.character(Time_start), "\\.", ":") |> str_trim(),
    Stop.time = str_replace_all(as.character(Stop.time), "\\.", ":") |> str_trim(),
    # Safe time parsing (NO WARNINGS)
    t_start = suppressWarnings(parse_date_time(Time_start, orders = c("H:M:S", "H:M"))),
    t_stop = suppressWarnings(parse_date_time(Stop.time, orders = c("H:M:S", "H:M"))),
    # Observation length
    Obs_length = as.numeric(t_stop - t_start, units = "secs"),
    # Timestamp
    TIMESTAMP = as.POSIXct(

```

```

    paste(Date, Time_start),
    format = "%m/%d/%Y %H:%M:%S",
    tz = "EST"
  ),
  # Sequential ID
  Seq = row_number(),
  # INAL standardized Plot label
  Plot = if_else(
    is.na(Letter),
    paste(Date, Site, Zone, Replicate, Seq, sep = "_"),
    paste(Date, Site, Zone, Replicate, Letter, Seq, sep = "_")) )

# Quick check
head(metadat$Plot)

```

4 Check start times overlaps or missing values

```

#checking if there is any start and stop time overlap from the field notes
overlaps_detected <- metadat %>%
  mutate(
    start_ts = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    ),
    end_ts = start_ts + Obs_length
  ) %>%
  arrange(SN, start_ts) %>%
  group_by(SN) %>%
  mutate(
    next_plot      = lead(Plot),
    next_start_ts  = lead(start_ts),
    overlap_fffi   = !is.na(next_start_ts) & next_start_ts <= end_ts
  ) %>%
  filter(overlap_fffi) %>%
  select(
    SN,
    Plot,
    start_ts,

```

```

    end_ts,
    next_plot,
    next_start_ts,
    Obs_length
  )

print(overlaps_detected)

#checking problems with obs_length - this is a frequent bug for manual field notes
field_notes_overlap<-metadat %>%
  mutate(
    obs_problem =
      is.na(Obs_length) | Obs_length <= 0
  ) %>%
  filter(obs_problem) %>%
  select(
    SN,
    Date,
    Plot,
    Time_start,
    Stop.time,
    Obs_length
  )

print(field_notes_overlap)

# checking unique measurment start time and stop time
# alldat %>%
#   filter(
#     SN == "TG10-01448",
#     TIMESTAMP > as.POSIXct("2023-01-05 13:28:00"),
#     TIMESTAMP < as.POSIXct("2023-01-05 13:33:00")
#   ) %>%
#   ggplot(aes(x = TIMESTAMP, y = CO2)) +
#   geom_point()#ylim(300,500)

```

5 Match metadata to raw data

```

#function to match data per equipment (considering different serial numbers) uhul!
match_flux_metadata <- function(alldat, metadat) {

```

```

message("Starting metadata  Licor matching")
#Sanity checks (fail early)
stopifnot(
  "TIMESTAMP" %in% names(alldat),
  "SN"         %in% names(alldat),
  "Date"       %in% names(metadat),
  "Time_start"%in% names(metadat),
  "Obs_length"%in% names(metadat),
  "Plot"       %in% names(metadat)
)

stopifnot(all(metadat$Obs_length > 0, na.rm = TRUE))
stopifnot(anyDuplicated(metadat$Plot) == 0)
# Instruments present in BOTH datasets
instruments <- intersect(
  unique(alldat$SN),
  unique(metadat$SN)
)
message("Instruments to process: ", paste(instruments, collapse = ", "))
results <- lapply(instruments, function(sn) {
  message("Processing SN: ", sn)
  dat_i <- alldat %>%
    filter(SN == sn)
  meta_i <- metadat %>%
    filter(SN == sn) %>%
    arrange(Date, Time_start) %>%
    mutate(metadat_row = row_number())
  stopifnot(nrow(meta_i) > 0)
# Actual matching
dat_i$metadat_row <- ffi_metadata_match(
  data_timestamps = dat_i$TIMESTAMP,
  start_dates     = meta_i$Date,
  start_times     = meta_i$Time_start,
  obs_lengths     = meta_i$Obs_length )
# Attach metadata (vectorized, fast)
dat_i <- dat_i %>%
  mutate(
    Plot      = meta_i$Plot[metadat_row],
    Site      = meta_i$Site[metadat_row],
    Zone      = meta_i$Zone[metadat_row],
    Volume    = meta_i$Volume[metadat_row],
    Temp      = meta_i$Temp[metadat_row],

```

```

        Area      = meta_i$Area[metadat_row],
        Dead_band = meta_i$Dead_band[metadat_row]      )
# Drop unmatched rows
n_before <- nrow(dat_i)
dat_i <- dat_i %>% filter(!is.na(metadat_row))
n_after <- nrow(dat_i)
message("  Matched rows: ", n_after, " / ", n_before)
dat_i })
alldat_matched <- rbindlist(results, fill = TRUE)
#Final sanity checks
stopifnot(
  !anyNA(alldat_matched$Plot),
  nrow(alldat_matched) > 0
)
message("Matching finished successfully")
return(alldat_matched)}

#running the funciton
message("Running final metadata matching...")
alldat_matched <- match_flux_metadata(alldat, metadat)
message("Matched dataset rows: ", nrow(alldat_matched))
head(alldat_matched)

#saving matched data and metadata as raw data is too big (today, Licor files leave in the CO2 column)
saveRDS(alldat_matched, file = "alldat_matched_2023b.rds")

#can remove any row that have NA's in the CO2 column
#alldat <- alldat %>%
# filter(!is.na(CO2))

```

6 reating alldata matched df

```

# after you match the data for the first time - we can just read the .rds file to save processing
# if you need to match licor data and metadata again, remove "#" from the previous chunks

alldat_matched <- readRDS("alldat_matched_2023b.rds")

```

```

##Extracting initial CO2 concentration

```



```
## Extract initial CO2 concentration (t0) for each Plot for QAQC later
#saving df to use after flux calculation
df_CO2_in <- alldat_matched %>%
  arrange(Plot, TIMESTAMP) %>%
  group_by(Plot) %>%
  summarise(CO2_initial = first(CO2), .groups = "drop")
```

7 Creating temperature csv from the L2-weather station in the wetland

```
# Getting Air temp from the weather station
## Read L2 temperature data from ZIP stored outside GitHub,
## extract desired variable, save lightweight CSV,
## and remove temporary unzipped files afterwards

## Read extracted L2 Parquet files and save temperature time series
BASE_DIR <- "D:/COMPASS_L2-v2-1/v2-1"
SITE <- "(OWC|CRC|PTR)"
VARIABLE <- "wx-tempavg15"

OUTFILE <- "Raw Data/L2_temp_timeseries.csv"

# Build regex that matches REAL L2 filenames
regex <- paste0(SITE, ".*", VARIABLE, ".*_L2_v2-1.*\\.parquet$")

# Locate Parquet files
files <- list.files(BASE_DIR, pattern = regex, recursive = TRUE, full.names = TRUE)

message("Files found: ", length(files))
print(basename(files))

# Fail fast if nothing is found
stopifnot(length(files) > 0)

# Read and combine Parquet files
dat <- lapply(files, function(f) {
  message("Reading ", basename(f))
  read_parquet(f)
})
```

```

dat <- bind_rows(dat)

# Optional QA filtering (only if column exists)
if ("QA_flag" %in% names(dat)) {
  dat <- dat %>% filter(QA_flag == 0)
}

# Keep only necessary columns
temps <- dat %>%
  select(TIMESTAMP, Plot, Site, Value) %>%
  rename(Temp = Value)

head(temps)

# Save lightweight output only
write.csv(
  temps,
  file = OUTFILE,
  row.names = FALSE
)

```

8 Reading Temp df to merge to metadat

```

#reading temp
temps <- read.csv("Raw Data/L2_temp_timeseries.csv")

## Match up the L1 temp data and add it to the metadata dataframe
#Going to need to use site and timestamp in the metadata in order to find a date

#make sure time stamps are in the correct format
alldat_matched$TIMESTAMP <- as.POSIXct(alldat_matched$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")
#problem here L2 data for midnight is always "2022-01-01" we loose this line to NA when form
temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)] <-
  paste0(temps$TIMESTAMP[!grepl(":", temps$TIMESTAMP)], " 00:00:00")
temps$TIMESTAMP <- as.POSIXct(temps$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")

#time rounding step
alldat_matched$TIMESTAMP_round <- round_date(alldat_matched$TIMESTAMP, "60 mins")
temps$TIMESTAMP_round <- round_date(temps$TIMESTAMP, "60 mins")

```

```
#ensure there is only one data observation per hour
temps_clean <- temps %>%
  group_by(TIMESTAMP_round, Site) %>%
  summarize(Temp = mean(Temp ))
```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by TIMESTAMP_round and Site.
- i Output is grouped by TIMESTAMP_round.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(TIMESTAMP\_round, Site))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

```
#join them
alldat_matched <- alldat_matched %>%
  left_join(temps_clean, by=c("Site", "TIMESTAMP_round"))
```

9 fixing some Deadbands and stop time

```
# Table of plots with custom deadband adjustments
time_overrides <- tibble::tibble(
  Plot = c("7/13/2023_CRC_TR_315_115",
           "7/27/2023_PTR_UP_8_125",
           "3/28/2023_OWC_TR_127_170",
           "5/10/2023_OWC_UP_103_73",
           "7/13/2023_CRC_TR_314_114",
           "8/3/2023_CRC_TR_314_133",
           "7/13/2023_CRC_TR_316_116",
           "6/29/2023_CRC_TR_311_104" ),
  dead_band_manual = c(8,9,22,16,8,16,3,34),
  stop_offset_secs = NA_real_ )

#apply new incubation period
alldat_trimmed <- alldat_matched %>%
  left_join(time_overrides, by = "Plot") %>%
  group_by(Plot) %>%
  mutate(
    t_start_original = min(TIMESTAMP),
    t_end_original   = max(TIMESTAMP),
```

```

# deadband manual or metadat
t_start_adjusted = if_else(
  !is.na(dead_band_manual),
  t_start_original + dead_band_manual,
  t_start_original + as.numeric(coalesce(first(Dead_band), 0))
),

# new offset
t_end_adjusted = if_else(
  !is.na(stop_offset_secs),
  t_end_original + stop_offset_secs,
  t_end_original
)
) %>%
filter(TIMESTAMP >= t_start_adjusted,
       TIMESTAMP <= t_end_adjusted) %>%
ungroup() %>%
select(-t_start_original, -t_end_original,
       -t_start_adjusted, -t_end_adjusted,
       -dead_band_manual, -stop_offset_secs)

# alldat_trimmed %>%
#   filter(Plot == "8/17/2022_OWC_UP_6_147") %>%
#   ggplot(aes(TIMESTAMP, CO2)) +
#   geom_point() + theme_classic()

```

10 Calculate fluxes

```

#change the units
alldat_trimmed$CO2_umol <- ffi_ppm_to_umol(alldat_trimmed$CO2,
                                           volume = alldat_trimmed$Volume, # m3
                                           temp = alldat_trimmed$Temp)      # degrees C from the weather

```

Assuming atm = 101325 Pa

Using R = 8.31446261815324 m3 Pa K-1 mol-1

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

13

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data

Error in lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...) :
0 (non-NA) cases

Warning in fit_function(x\$.norm_time, x[, gas_column], ...): Could not fit
linear model

Error in lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...) :
0 (non-NA) cases

Warning in fit_function(x\$.norm_time, x[, gas_column], ...): Could not fit
linear model

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data
NOTE: HM81_flux.estimate is not NA, implying nonlinear data

[illegible]

[illegible]

[illegible]

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data

```
head(fluxes)
```

	Plot	TIMESTAMP	HM81_AIC	HM81_flux.estimate
1	1/12/2023_PTR_TR_1_26	2023-01-12 11:25:29	-1673.2260	0.001488583
2	1/12/2023_PTR_TR_2_27	2023-01-12 11:41:49	-373.3861	0.003853974
3	1/12/2023_PTR_TR_3_28	2023-01-12 11:55:29	-874.8432	0.002337582
4	1/12/2023_PTR_TR_4_29	2023-01-12 12:07:29	-303.6624	-0.006803087
5	1/12/2023_PTR_TR_4_B_30	2023-01-12 12:18:29	-1578.3103	0.001503884
6	1/12/2023_PTR_TR_5_31	2023-01-12 12:26:29	-807.8274	0.003413364

	HM81_p.value	HM81_r.squared	HM81_RMSE	lin_AIC	lin_flux.estimate
1	7.666169e-156	0.6938552	0.05980751	-3046.562	0.001226571
2	4.580077e-198	0.6957943	0.18877166	-3577.708	0.003108942
3	1.274356e-142	0.6610713	0.11633224	-3306.504	0.002296731
4	7.782742e-168	0.7208468	0.18724805	-1774.056	-0.004036002
5	1.376542e-143	0.6635825	0.06473018	-3148.011	0.001270600
6	4.004211e-161	0.7060499	0.12301382	-2749.865	0.002600406

	lin_flux.std.error	lin_int.estimate	lin_int.std.error	lin_p.value
1	8.976872e-06	15.63384	0.001550952	0
2	7.582276e-06	15.30115	0.001660215	0
3	7.228508e-06	15.50874	0.001248884	0
4	2.592151e-05	19.25761	0.004478512	0
5	8.249156e-06	15.54712	0.001425223	0
6	1.149485e-05	15.44359	0.001985988	0

	lin_r.squared	lin_RMSE	poly_AIC	poly_r.squared	poly_RMSE	rob_AIC
1	0.9689635	0.01904272	-3200.647	0.9761523	0.01672027	-3046.237
2	0.9955116	0.02292967	-4132.399	0.9978481	0.01589799	-3575.361

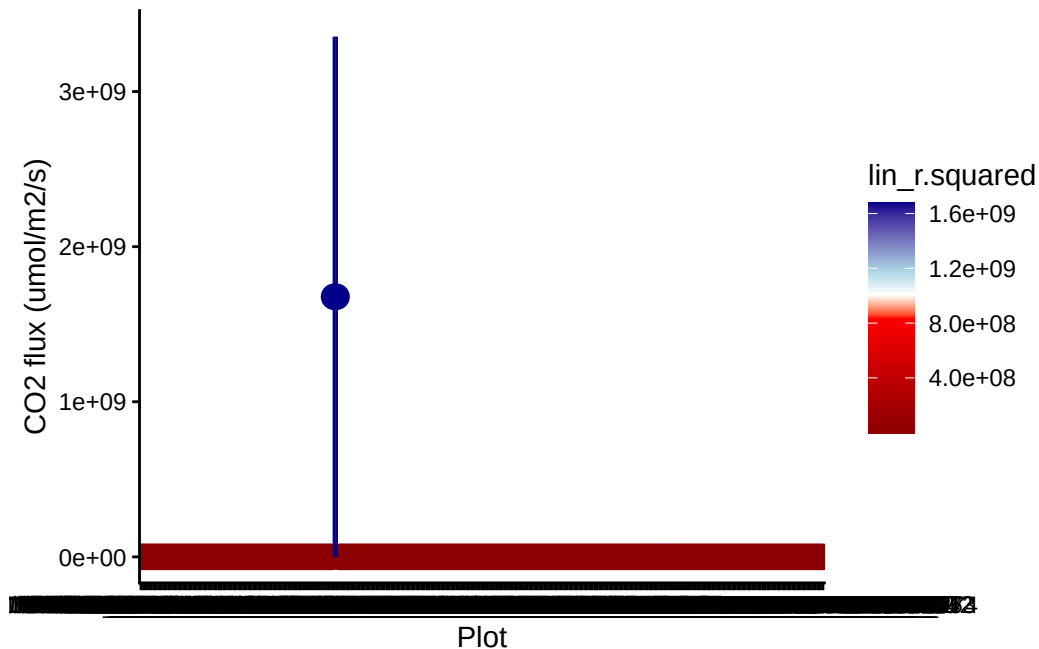
3	0.9941114	0.01533390	-3317.696	0.9942586	0.01516638	-3306.277
4	0.9759267	0.05498753	-2564.355	0.9935938	0.02841353	-1774.043
5	0.9754138	0.01749901	-3216.570	0.9782143	0.01649987	-3147.846
6	0.9884500	0.02438412	-3222.624	0.9947822	0.01641684	-2749.373

	rob_converged	rob_flux.estimate	rob_flux.std.error	rob_RMSE
1	1	0.001221808	9.771540e-06	0.02051275
2	1	0.003119893	7.855219e-06	0.02148676
3	1	0.002299814	7.665480e-06	0.01546289
4	1	-0.004034655	2.673182e-05	0.06949846
5	1	0.001269843	8.180208e-06	0.01602977
6	1	0.002594641	1.180629e-05	0.02575103

	TIMESTAMP_min		TIMESTAMP_max	
1	2023-01-12	11:23:00	2023-01-12	11:27:59
2	2023-01-12	11:38:40	2023-01-12	11:44:59
3	2023-01-12	11:53:00	2023-01-12	11:57:59
4	2023-01-12	12:05:00	2023-01-12	12:09:59
5	2023-01-12	12:16:00	2023-01-12	12:20:59
6	2023-01-12	12:24:00	2023-01-12	12:28:59

```
#merging fluxes with initial CO2 concentration
fluxes <- fluxes %>%
  left_join(df_CO2_in, by = "Plot")

#take a look at the fluxes we calculated with R2
ggplot(fluxes, aes(Plot, lin_flux.estimate, color = lin_r.squared)) +
  geom_point(size=4) + theme_classic() +
  geom_linerange(aes(ymin = lin_flux.estimate- lin_flux.std.error,
                    ymax = lin_flux.estimate+ lin_flux.std.error)) +
  scale_color_gradientn(colours = c("darkred", "red", "white", "lightblue", "darkblue"),
                       values = c(0, 0.5, 0.6, 0.7, 1)) +
  ylab("CO2 flux (umol/m2/s)")
```



```
#sort fluxes by R2
fluxes_good <- as.data.frame(subset(fluxes, lin_r.squared >= .90, select = Plot:TIMESTAMP_ma
fluxes_lowr2 <- as.data.frame(subset(fluxes, lin_r.squared < .90 & lin_flux.estimate < 1, se
fluxes_weird <- as.data.frame(subset(fluxes, lin_r.squared < .90, select = Plot:TIMESTAMP_ma

#find fluxes that represent the first and fourth quartile of the data
low <- quantile(fluxes$lin_flux.estimate, 0.05)
high <- quantile(fluxes$lin_flux.estimate, 0.95)

fluxes_low <- as.data.frame(subset(fluxes, lin_flux.estimate <= low, select = Plot:TIMESTAMP
fluxes_high <- as.data.frame(subset(fluxes, lin_flux.estimate >= high, select = Plot:TIMESTAI

tails <- rbind(fluxes_low, fluxes_high)
```

11 check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)

```
# grab trim limits for each overridden Plot
trim_limits <- alldat_trimmed %>%
```

```

filter(Plot %in% time_overrides$Plot) %>%
group_by(Plot) %>%
summarise(
  t_start_trim = min(TIMESTAMP),
  t_end_trim   = max(TIMESTAMP) )

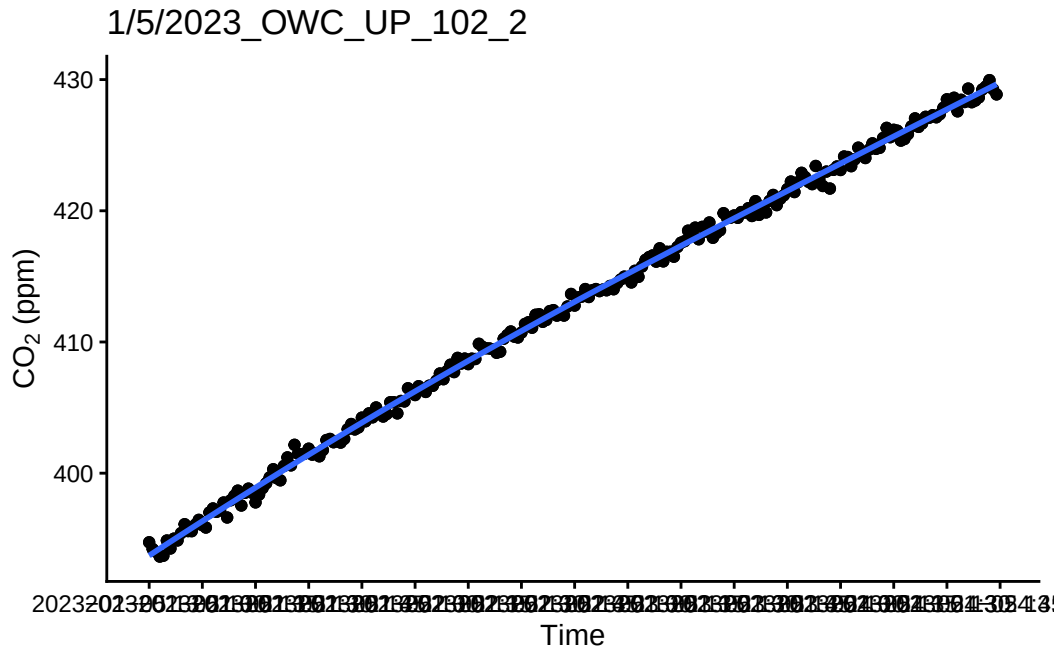
# use full alldat_matched + trim limits to classify each point
r_all <- alldat_matched %>%
  filter(Plot %in% time_overrides$Plot) %>%
  left_join(trim_limits, by = "Plot") %>%
  group_by(Plot) %>%
  mutate(
    time_sec = as.numeric(difftime(TIMESTAMP, min(TIMESTAMP), units = "secs")),
    point_status = case_when(
      TIMESTAMP < t_start_trim ~ "Discarded (deadband)",
      TIMESTAMP > t_end_trim   ~ "Discarded (stop trim)",
      TRUE                     ~ "Used"   )) %>%
  ungroup()

# plot all overridden plots
ggplot(r_all, aes(x = TIMESTAMP, y = CO2, color = point_status)) +
  geom_point(size = 1.8) +
  facet_wrap(~ Plot, scales = "free", ncol = 2) +
  scale_color_manual(
    values = c(
      "Used" = "#E41A1C",
      "Discarded (deadband)" = "grey60",
      "Discarded (stop trim)" = "grey20"
    )
  ) +
  theme_classic() +
  ylab(expression(paste(CO[2], " (ppm)"))) +
  xlab("Time") +
  labs(
    title = "Incubation Period Adjustments",
    color = "Point Status") +
  theme(legend.position = "bottom")

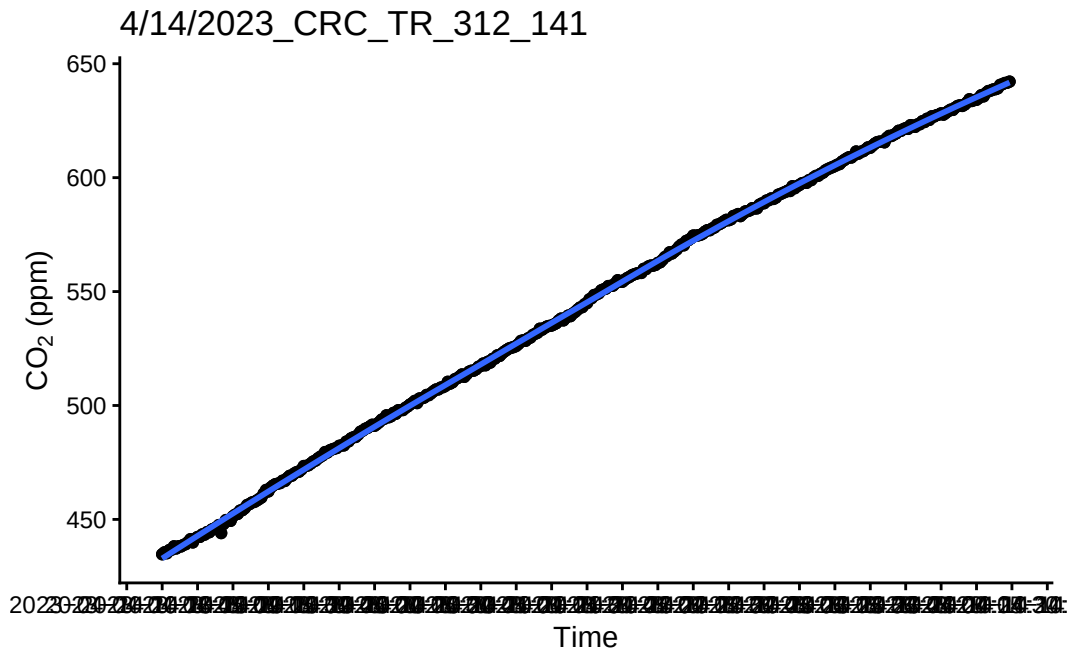
```

12 Spot check a few fluxes with $R^2 > 0.9$

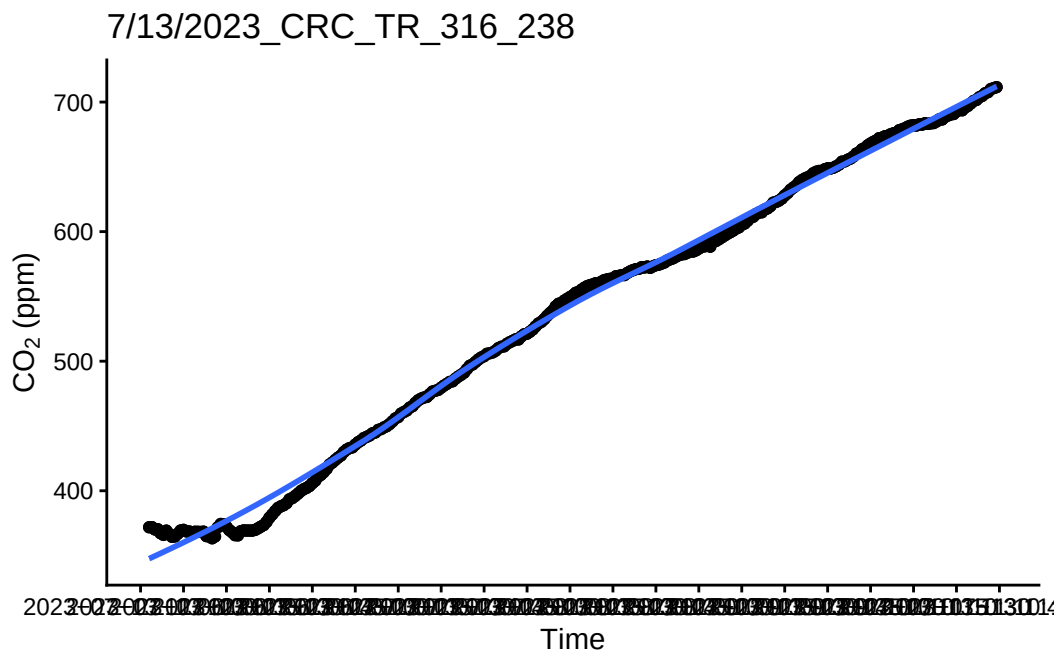
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



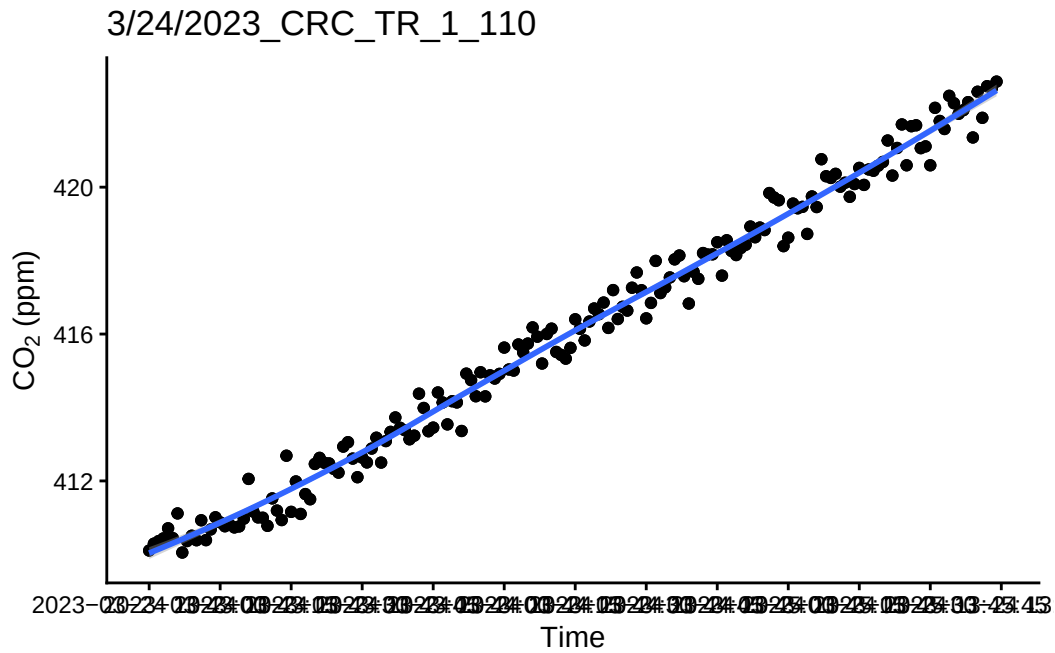
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



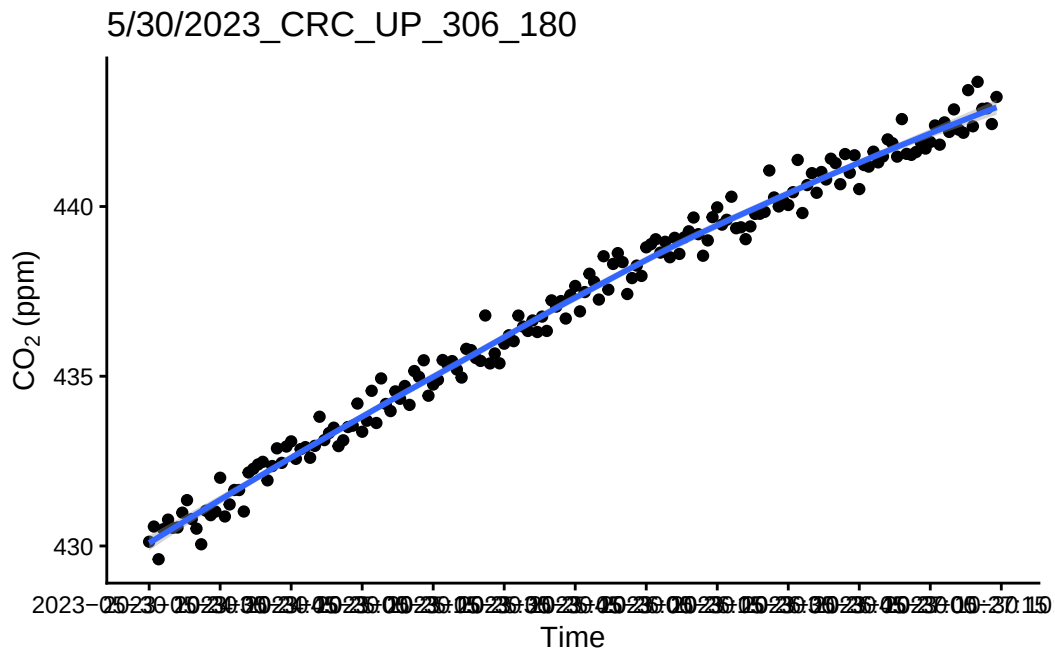
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



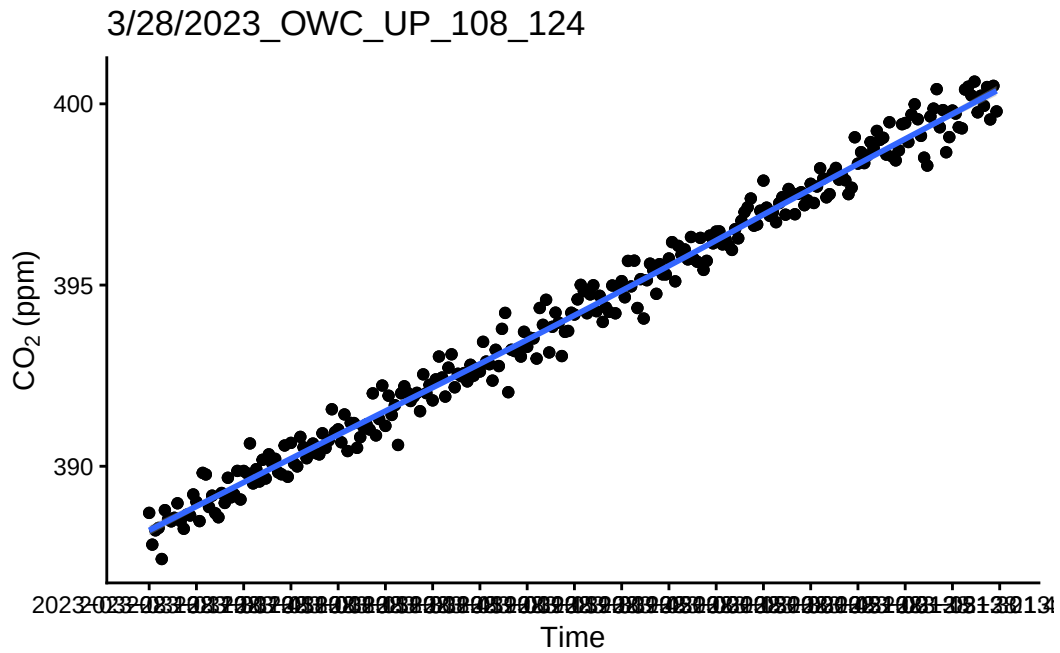
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



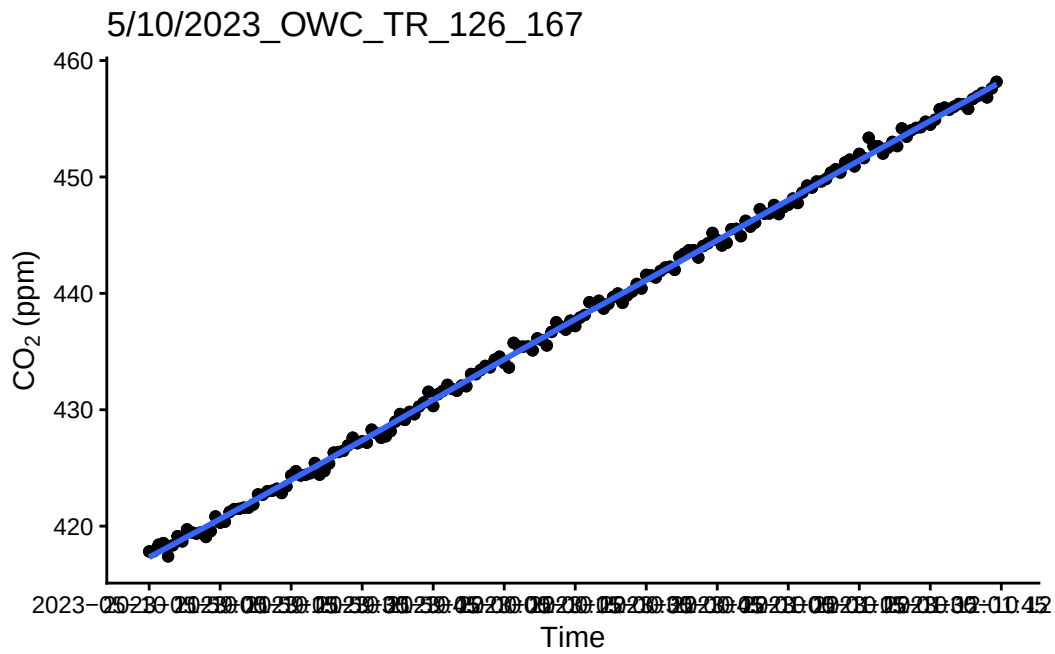
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



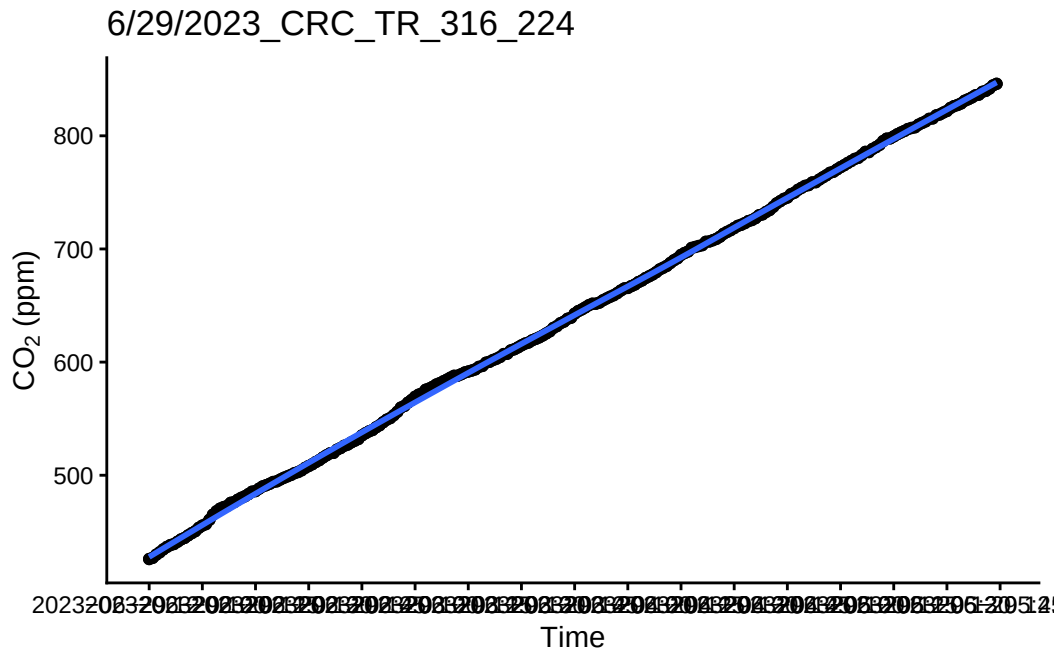
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



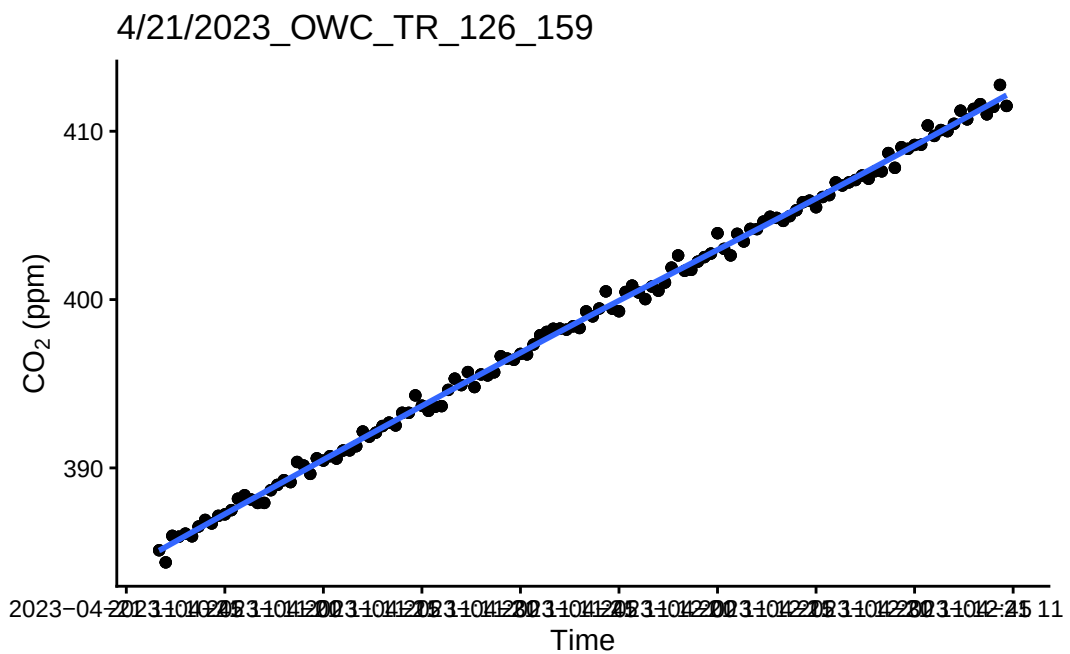
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

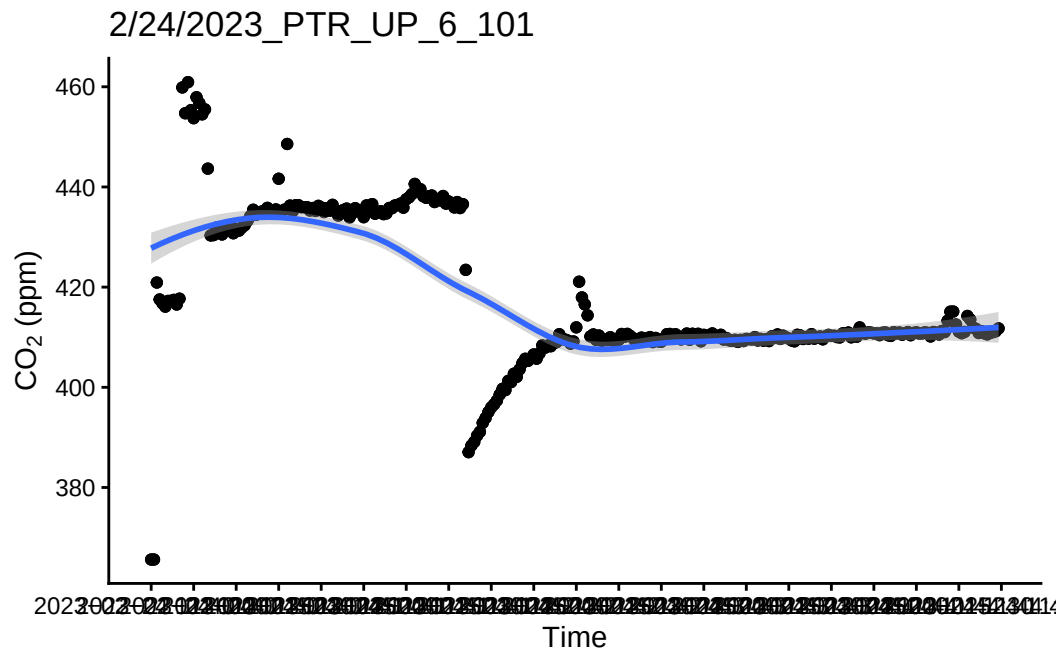


``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

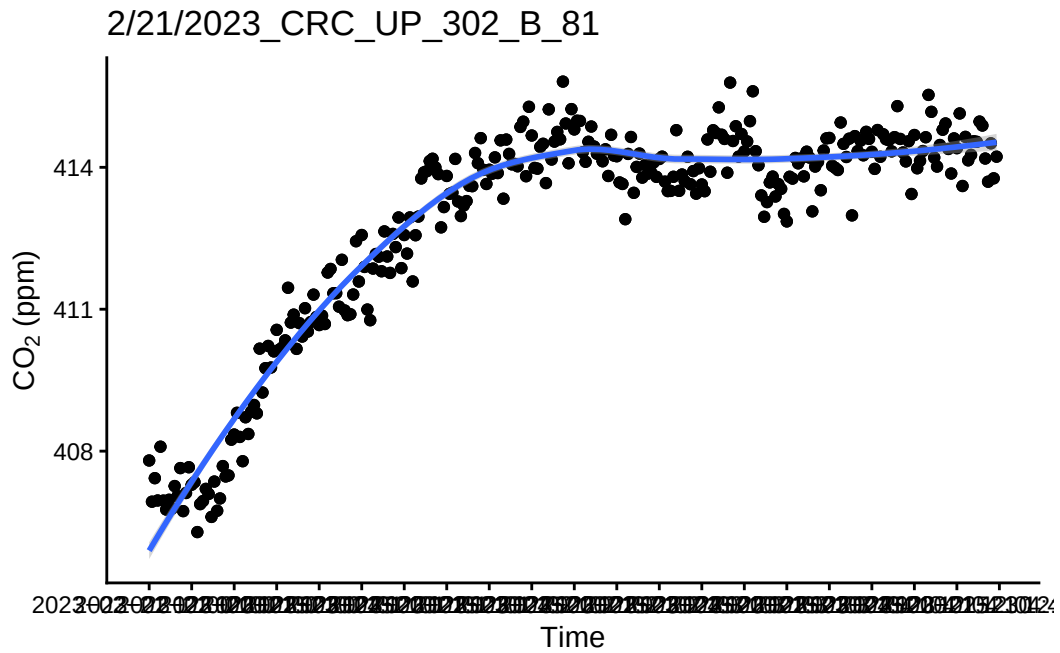


13 Take a look at the fluxes < 0.9 and a flux estimate below 1

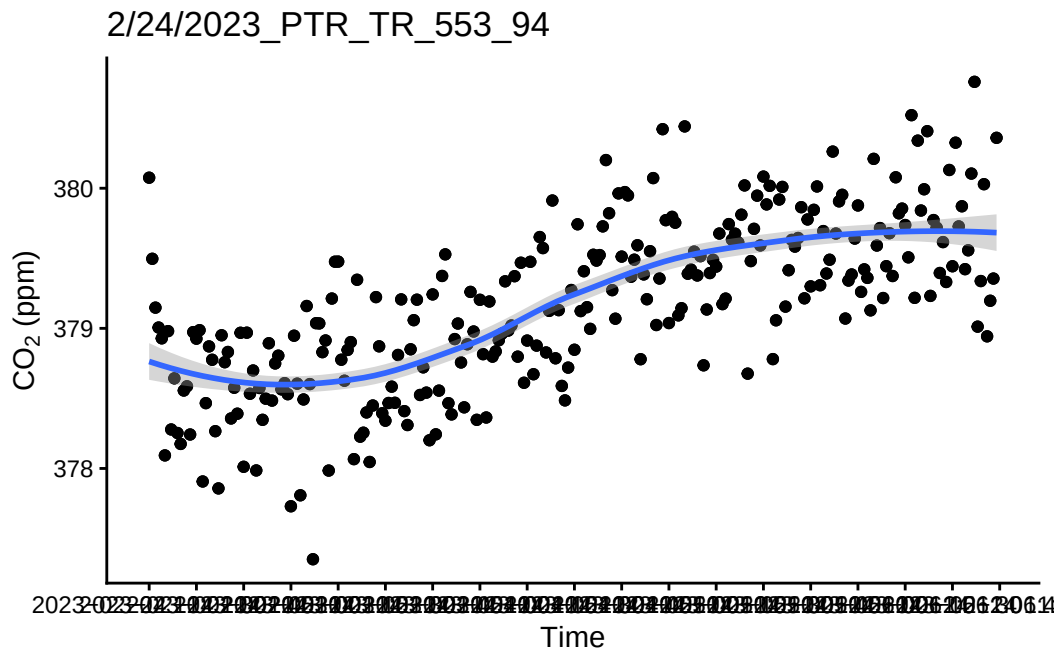
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



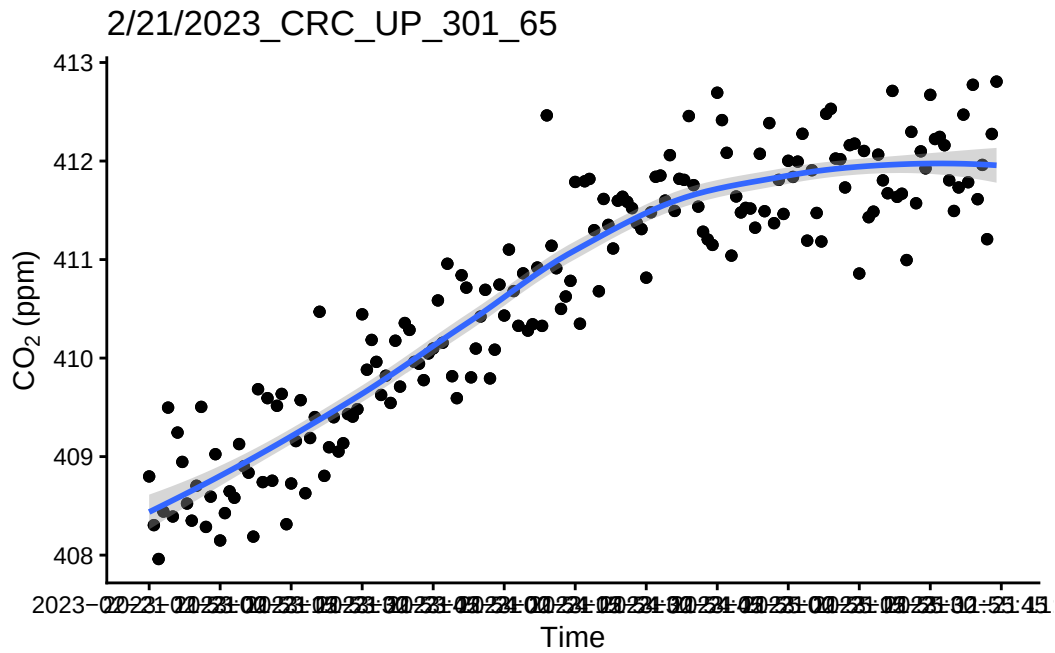
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



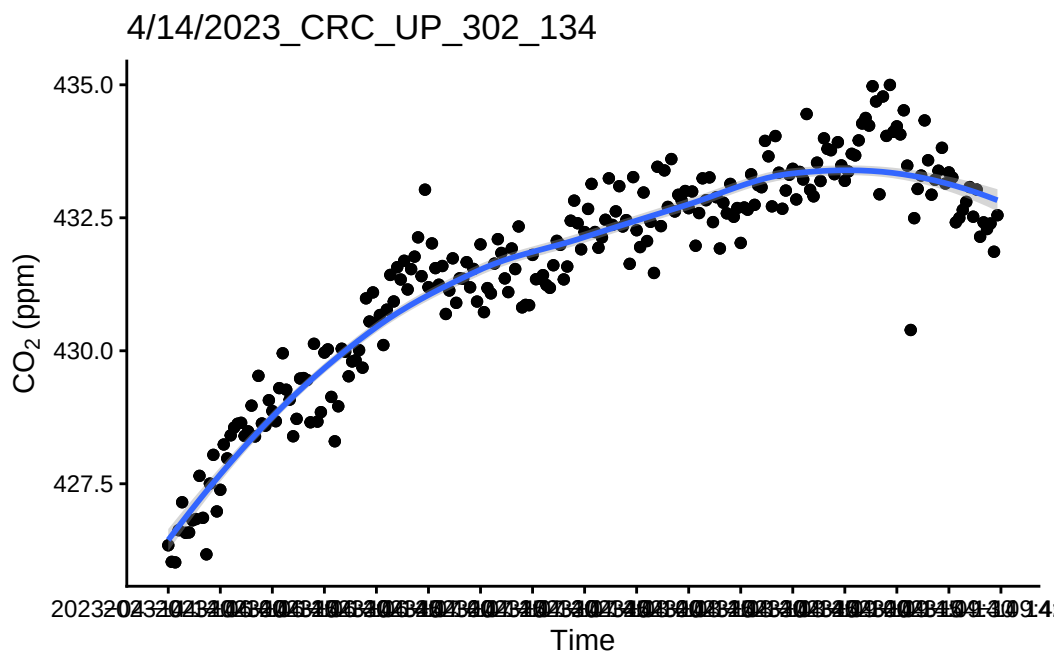
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



14 Take a look at the bad fluxes < 0.9 and have a high flux estimate

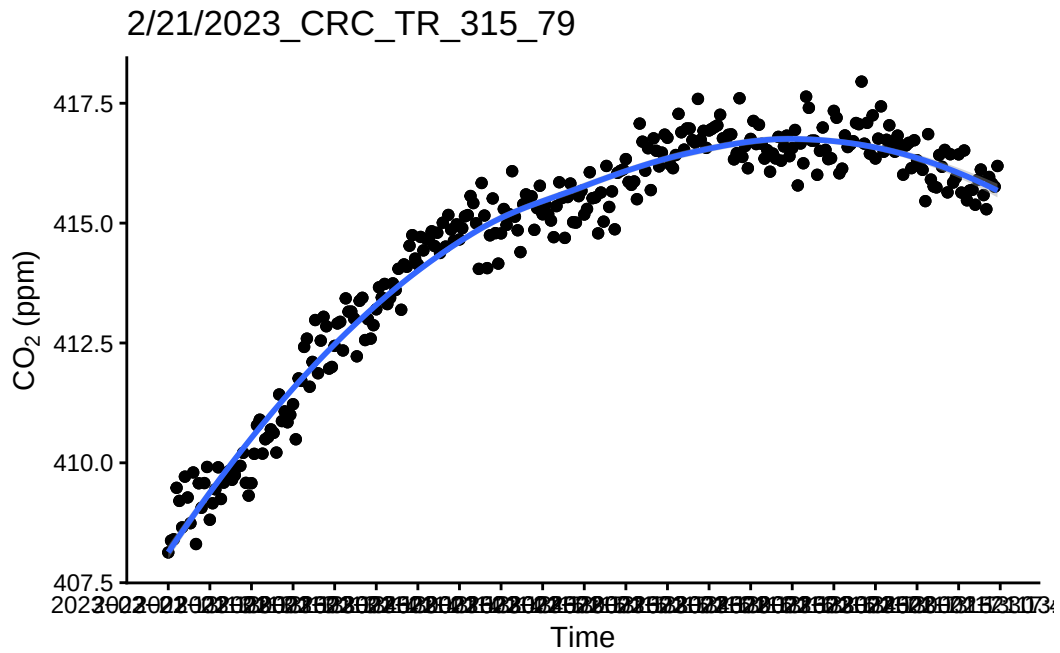
15 See if we want to remove these fluxes or if we want to try and recalculate these ones?

```
### pull out fluxes with R2 < 0.9 and put them in a list
bad_plots <- unique(fluxes_weird$Plot)

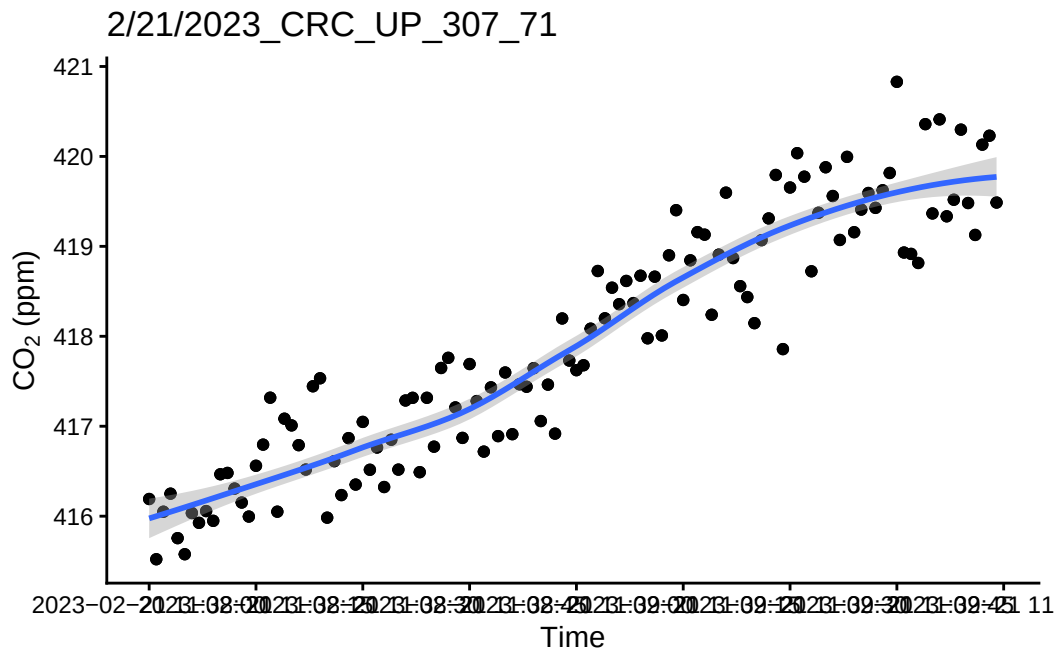
# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
bad_sample <- sample(bad_plots, 5, replace=FALSE)

#Loop to run five random plots subset above
for(P in bad_sample){
  r1 <- subset(alldat_trimmed, Plot == P & !is.na(metadata_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO[2], " (ppm)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
  print(p1)
}
```

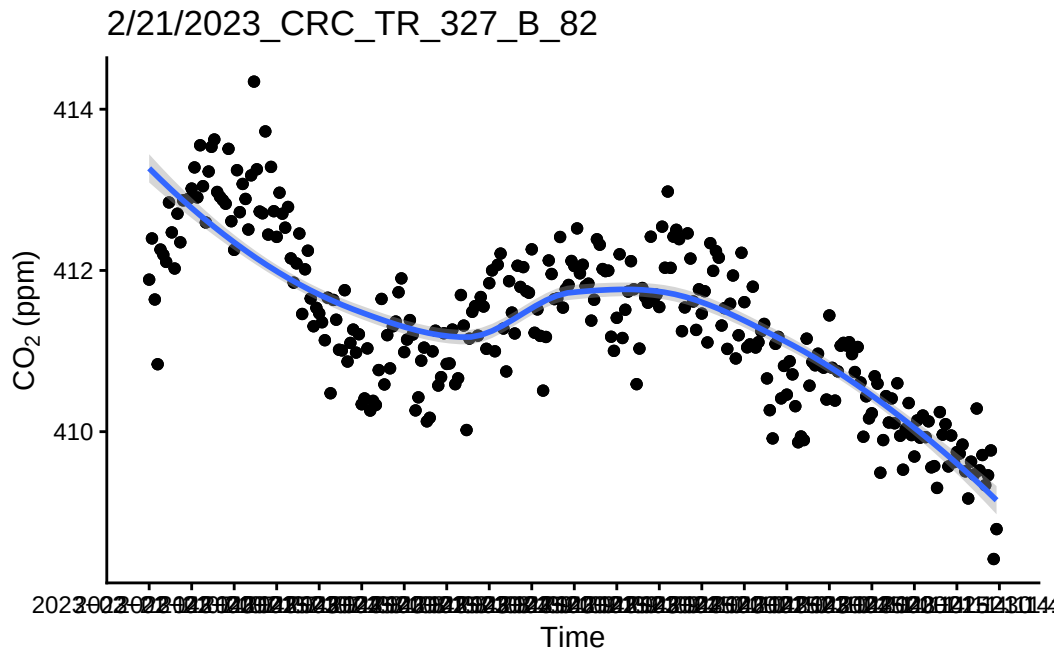
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



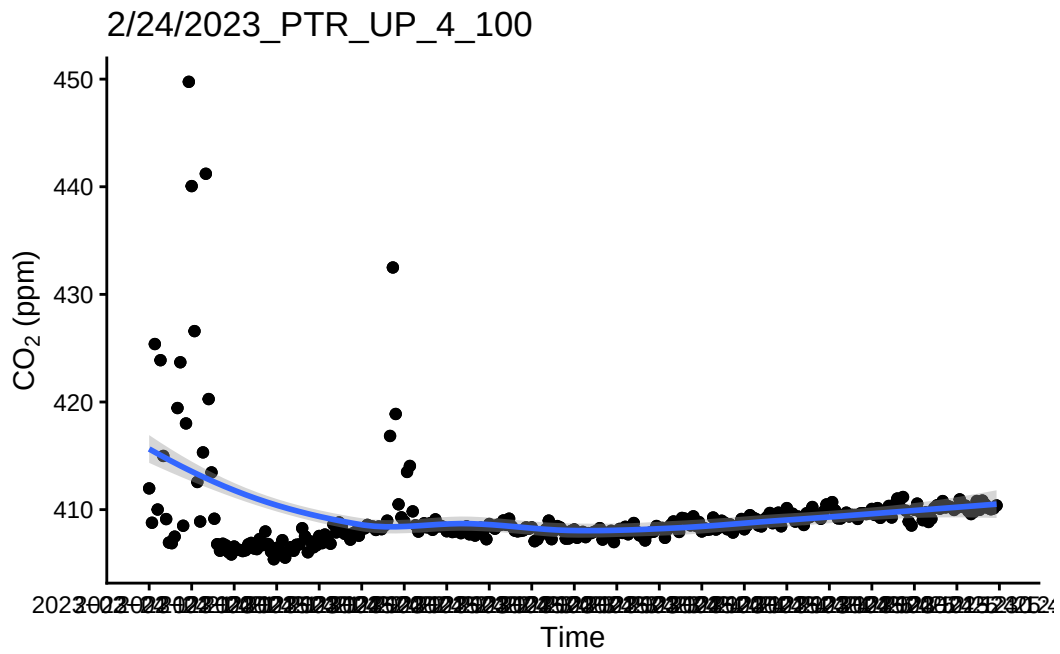
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



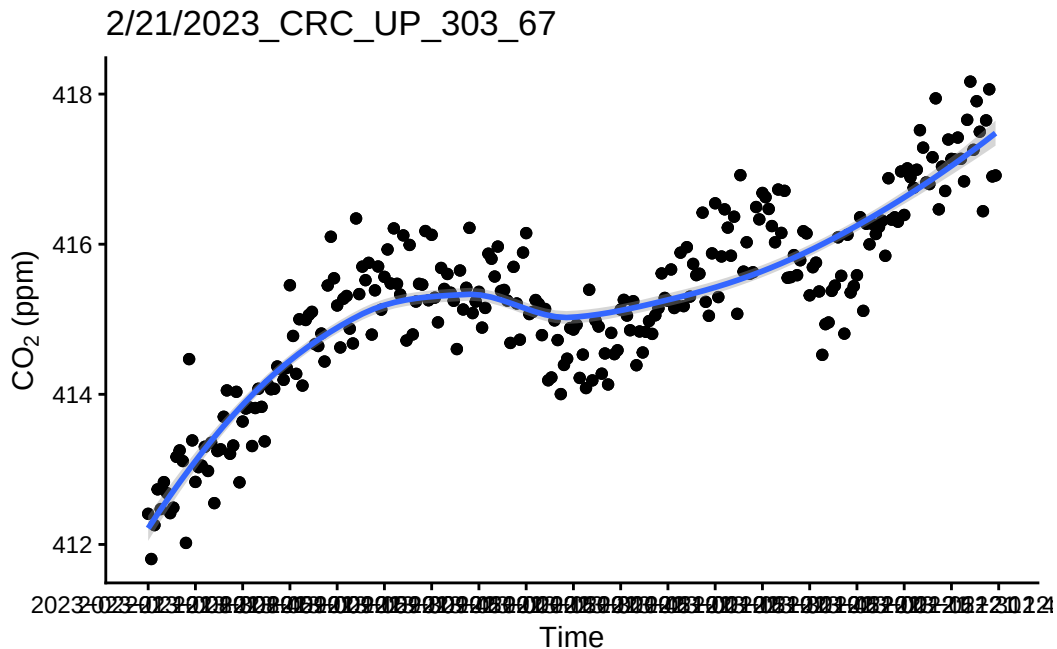
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()` using method = 'loess' and formula = 'y ~ x'`



``geom_smooth()` using method = 'loess' and formula = 'y ~ x'`



#####

16 Look at the really high or really low ones to see if there is a timing issue or something

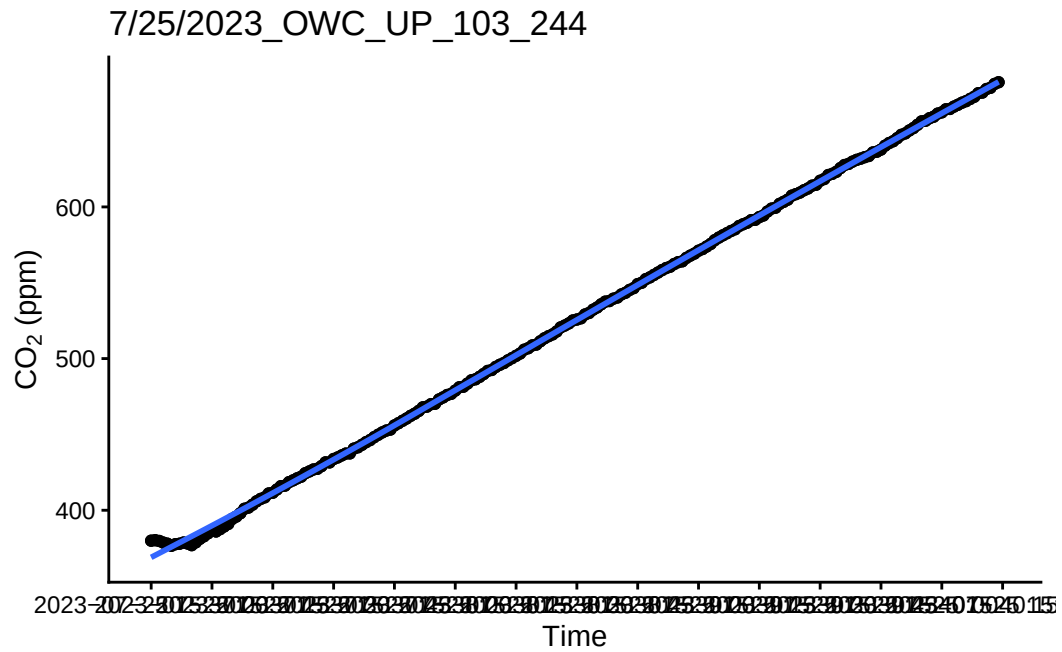
```
### pull out fluxes in the 5% and 95% tails to look at really high and really low
tail_plots <- unique(tails$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
tail_sample <- sample(tail_plots, 2, replace=FALSE)

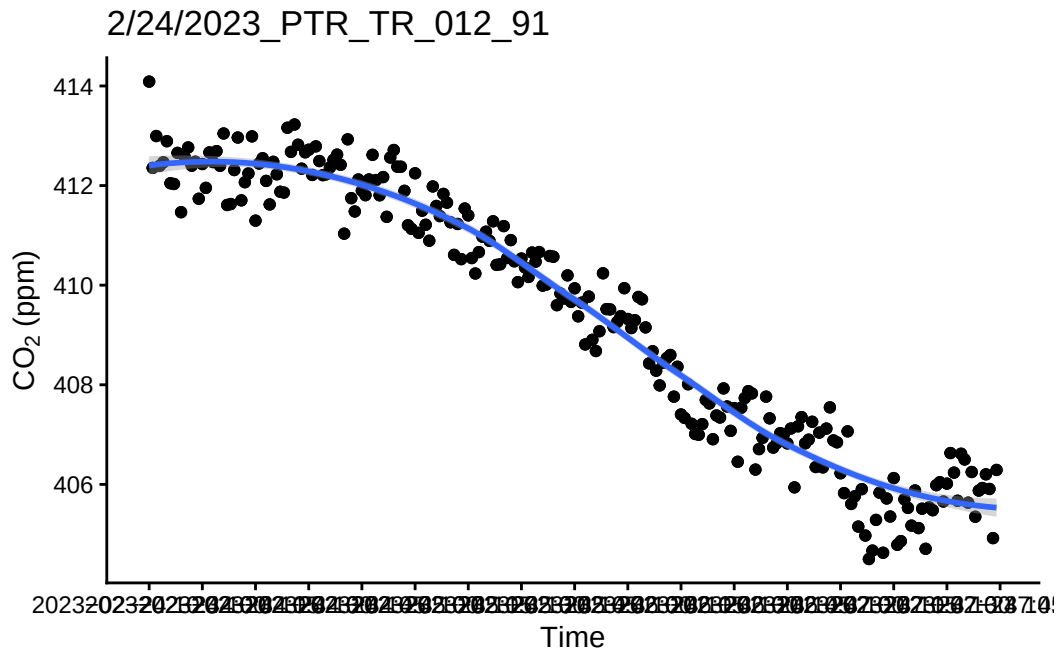
#Loop to run five random plots subset above
for(P in tail_sample){
  r1 <- subset(alldat_trimmed, Plot == P & !is.na(metadata_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO [2], " (ppm)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
```

```
print(p1)
}
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



```
#would be nice to put on the plot the flux estimate and if it is a high or low or something
#####
```

17 Check all fluxes

```
##Put all fluxes together
#fluxes_all <- as.data.frame(subset(fluxes, select = Plot:TIMESTAMP_max))

all_plots <- unique(fluxes$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
all_sample <- sample(all_plots, 5, replace=FALSE)

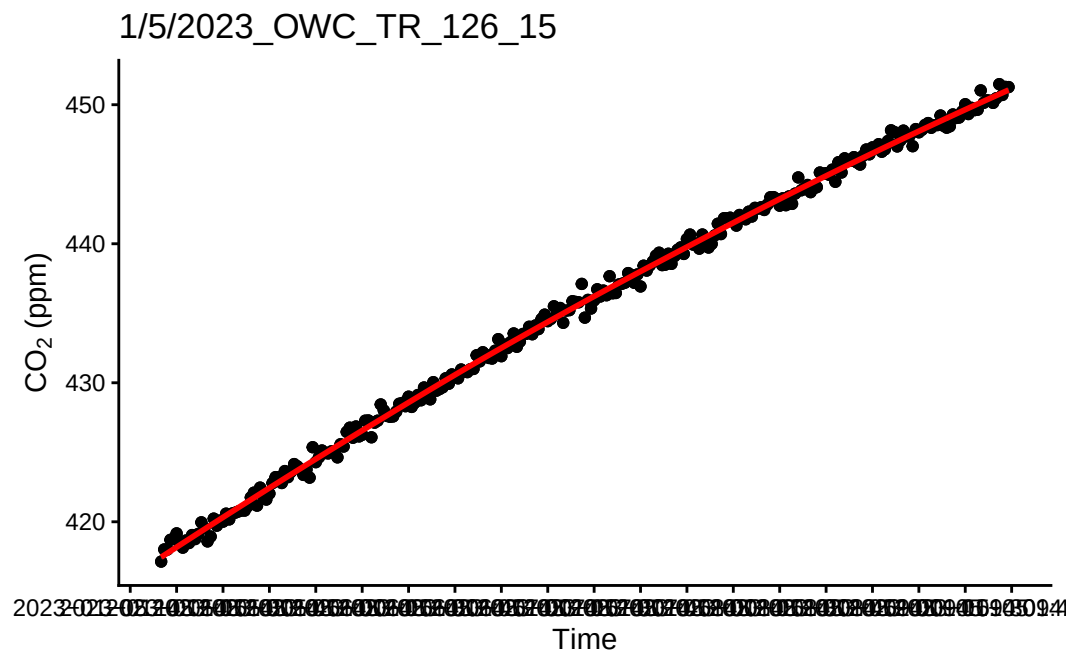
#Loop to run five random plots subset above
for(P in all_sample){
  r1 <- subset(alldat_trimmed, Plot== P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO [2], " (ppm)"))) +
```

```

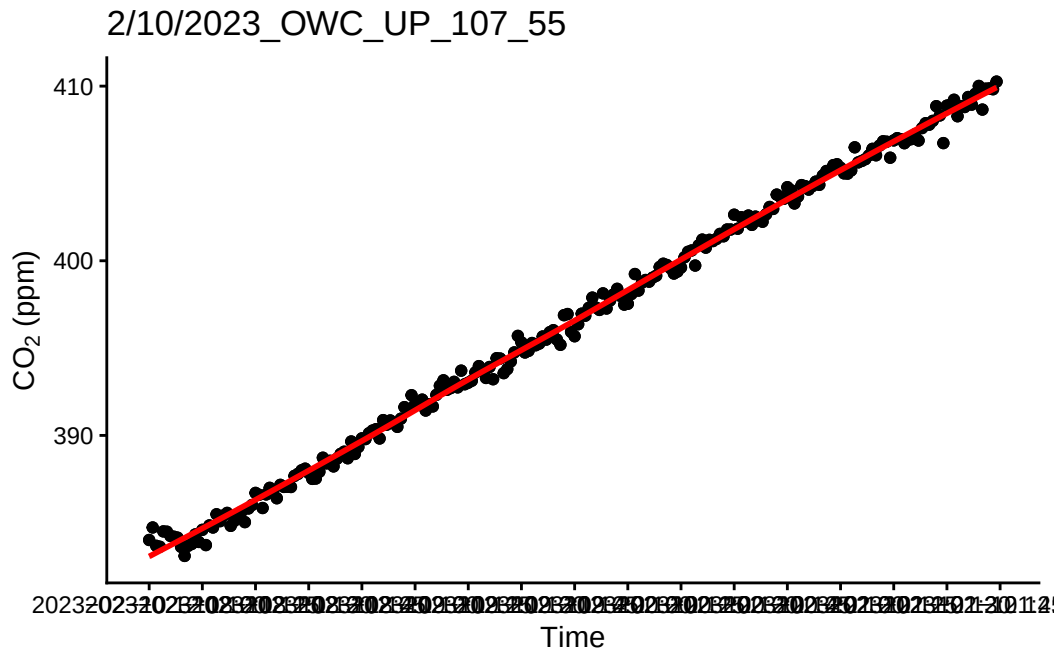
xlab("Time") + labs(title = P) +
geom_smooth(color="red")
print(p1)
}

```

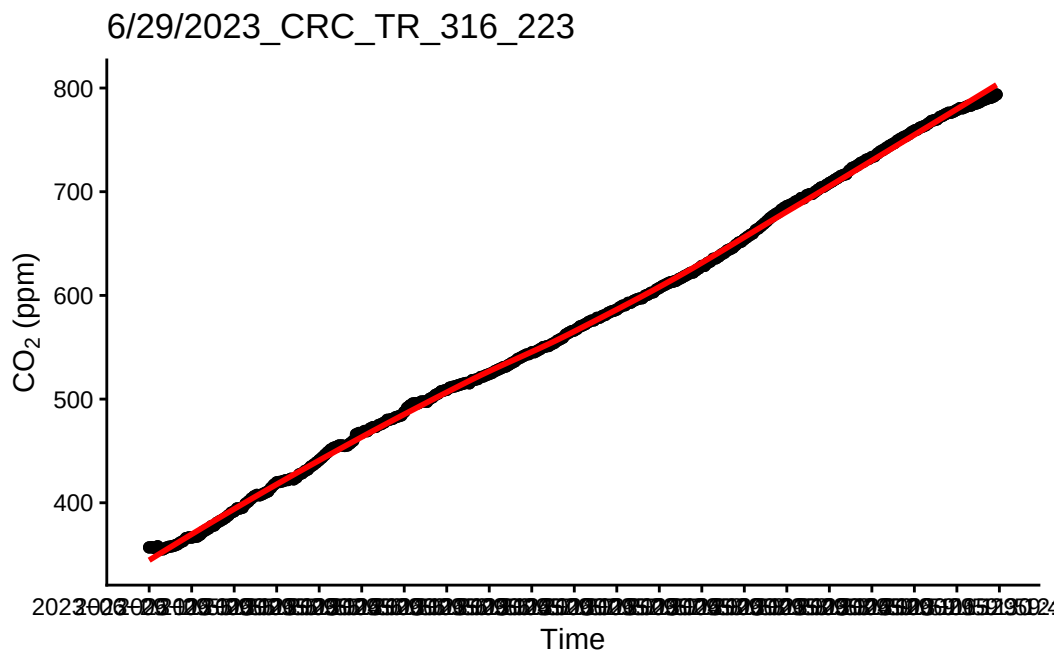
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



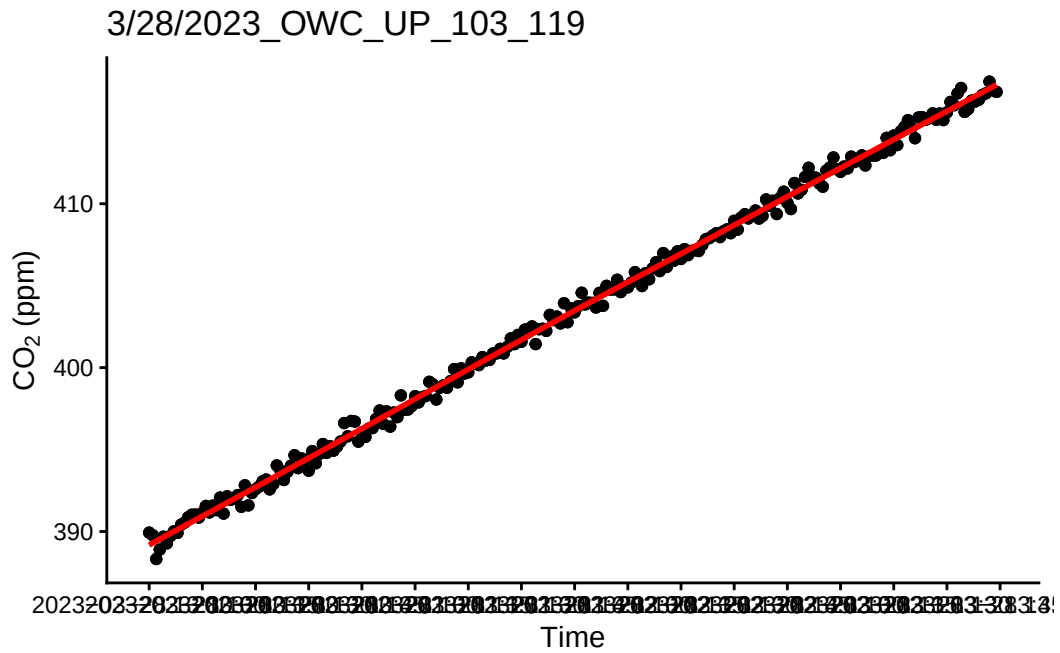
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



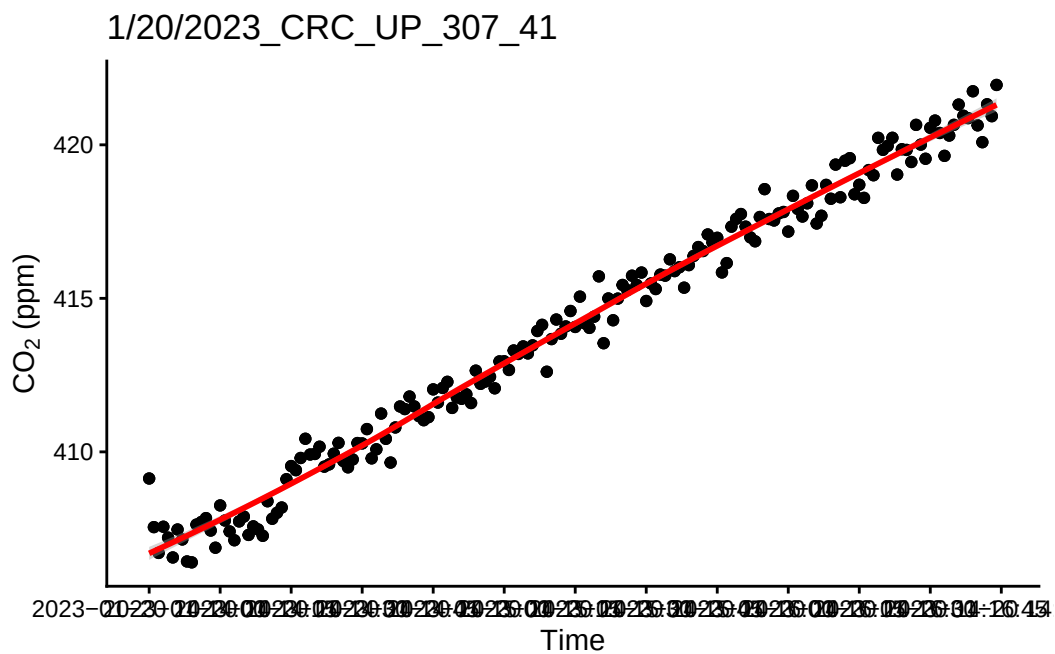
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



```
#####
```

18 QAQC - Flags

```
# QAQC flags
fluxes_flagged <- fluxes %>%
  mutate(
    # model fit flags
    CO2_lin_pass = (lin_r.squared >= 0.85 | lin_RMSE < 5) & !is.na(lin_r.squared),
    CO2_HM81_pass = (HM81_r.squared >= 0.85 | HM81_RMSE < 5) & !is.na(HM81_r.squared),
    CO2_poly_pass = (poly_r.squared >= 0.85 | poly_RMSE < 5) & !is.na(poly_r.squared),
    # negative flux flag
    CO2_neg_flux = lin_flux.estimate < 0 | is.na(lin_flux.estimate),
    # rejection flags
    CO2_rejected_model = !(CO2_lin_pass | CO2_HM81_pass | CO2_poly_pass),
    CO2_rejected_negflux = CO2_neg_flux,
    CO2_flux_rejected = CO2_rejected_model | CO2_rejected_negflux
  )

# percentage of problems
r2_summary <- fluxes_flagged %>%
  summarise(
    CO2_total_obs = n(),
    CO2_n_rejected_model = sum(CO2_rejected_model, na.rm = TRUE),
    CO2_n_rejected_negflux = sum(CO2_rejected_negflux, na.rm = TRUE),
    CO2_n_rejected_both = sum(CO2_rejected_model & CO2_rejected_negflux, na.rm = TRUE),
    CO2_n_rejected_total = sum(CO2_flux_rejected, na.rm = TRUE),
    CO2_pct_rejected_total = 100 * CO2_n_rejected_total / CO2_total_obs,
    CO2_n_lin_pass = sum(CO2_lin_pass, na.rm = TRUE),
    CO2_n_HM81_pass = sum(CO2_HM81_pass, na.rm = TRUE),
    CO2_n_poly_pass = sum(CO2_poly_pass, na.rm = TRUE)
  )

# formatted table for PDF
r2_summary %>%
  tidyr::pivot_longer(everything(), names_to = "Metric", values_to = "Value") %>%
  mutate(
    Metric = recode(Metric,
      CO2_total_obs = "Total observations",
      CO2_n_rejected_model = "Rejected - poor model fit (n)",
```

```

CO2_n_rejected_negflux = "Rejected - negative CO2 flux (n)",
CO2_n_rejected_both    = "Rejected - both reasons (n)",
CO2_n_rejected_total   = "Rejected - total (n)",
CO2_pct_rejected_total = "Rejected - total (%)",
CO2_n_lin_pass         = "Passed - Linear model (n)",
CO2_n_HM81_pass        = "Passed - HM81 model (n)",
CO2_n_poly_pass        = "Passed - Polynomial model (n)"
),
Value = ifelse(Metric == "Rejected - total (%)",
               round(Value, 1),
               as.integer(Value)) ) %>%

kable(
  caption = "QA/QC CO2 Flux Summary",
  align   = c("l", "r"),
  booktabs = TRUE,
  format   = "latex") %>%
kable_styling(
  latex_options = c("striped", "hold_position"),
  full_width    = FALSE ) %>%
pack_rows("Rejections",      2, 6) %>%
pack_rows("Model performance", 7, 9)

```

Table 1: QA/QC CO2 Flux Summary

Metric	Value
Total observations	273.0
Rejections	
Rejected — poor model fit (n)	0.0
Rejected — negative CO2 flux (n)	6.0
Rejected — both reasons (n)	0.0
Rejected — total (n)	6.0
Rejected — total (%)	2.2
Model performance	
Passed — Linear model (n)	273.0
Passed — HM81 model (n)	203.0
Passed — Polynomial model (n)	273.0

```

# R2 vs RMSE diagnostic plot
# pivot_longer keeps R2/RMSE columns and uses CO2*_pass as the pass indicator
set.seed(42)

```



```

plot_data <- fluxes_flagged %>%
  slice_sample(n = min(15000, nrow(.))) %>%
  select(
    lin_r.squared, lin_RMSE, CO2_lin_pass,
    HM81_r.squared, HM81_RMSE, CO2_HM81_pass,
    poly_r.squared, poly_RMSE, CO2_poly_pass,
    CO2_rejected_negflux
  ) %>%
  # rename pass columns to match model prefix before pivoting
  rename(
    lin_pass = CO2_lin_pass,
    HM81_pass = CO2_HM81_pass,
    poly_pass = CO2_poly_pass
  ) %>%
  pivot_longer(
    cols = -CO2_rejected_negflux,
    names_to = c("model", ".value"),
    names_pattern = "(lin|HM81|poly)_(.*)"
  ) %>%
  rename(r2 = r.squared) %>%
  mutate(
    model = recode(model, lin = "Linear", HM81 = "HM81", poly = "Polynomial"),
    status = case_when(
      CO2_rejected_negflux ~ "Negative CO2",
      pass ~ "Accepted",
      TRUE ~ "Rejected"
    )
  ) %>%
  filter(!is.na(r2), !is.na(RMSE))

#graphing
QAQC_plot <- ggplot(plot_data, aes(x = r2, y = RMSE, color = model, shape = status)) +
  geom_point(size = 2) +
  geom_vline(xintercept = 0.85, linetype = "dashed", color = "black", linewidth = 0.8) +
  geom_hline(yintercept = 5, linetype = "dashed", color = "black", linewidth = 0.8) +
  annotate("text", x = 0.7, y = max(plot_data$RMSE, na.rm = TRUE) * 0.95,
    label = "R2 = 0.85", hjust = -0.1, size = 3.5, fontface = "bold") +
  annotate("text", x = 0.1, y = 5,
    label = "RMSE = 5", vjust = -0.5, size = 3.5, fontface = "bold") +
  scale_color_manual(values = c(
    "Linear" = "#E41A1C",
    "HM81" = "#377EB8",

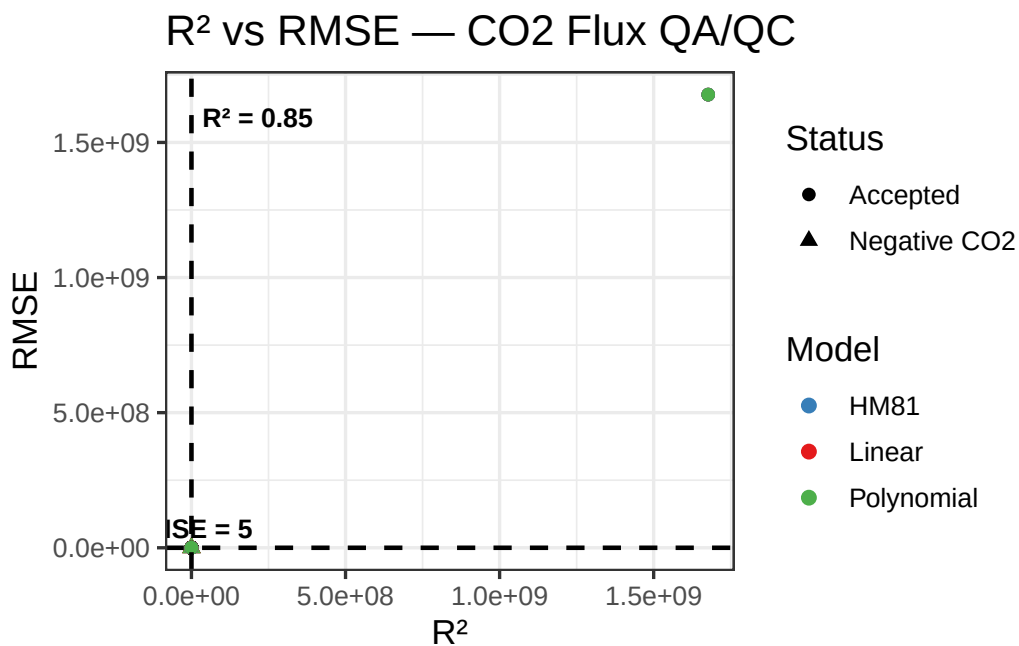
```

```

    "Polynomial" = "#4DAF4A"
  )) +
  scale_shape_manual(values = c(
    "Accepted"      = 16,
    "Rejected"      = 4,
    "Negative CO2" = 17 )) +
  labs(
    title = "R2 vs RMSE - CO2 Flux QA/QC",
    x      = "R2",
    y      = "RMSE",
    color  = "Model",
    shape  = "Status" ) +
  theme_bw(base_size = 13) +
  theme(legend.position = "right")

print(QAQC_plot)

```



19 Adding manual flag to reject any fluxes that do not fit criteria

```

# adding manual flags to the ones that are crazy off based on visual checks
manual_reject <- c(
)

fluxes_flagged <- fluxes_flagged %>%
  mutate(
    CO2_rejected_manual = Plot %in% manual_reject,
    CO2_flux_rejected = CO2_rejected_model | CO2_rejected_negflux | CO2_rejected_manual)

# fluxes_flagged %>%
#   filter(CO2_rejected_manual) %>%
#   select(Plot, lin_r.squared, lin_RMSE, CO2_rejected_model, CO2_rejected_negCO2, CO2_rejected)

```

20 Do some visualization of QAQC plots to assess fluxes

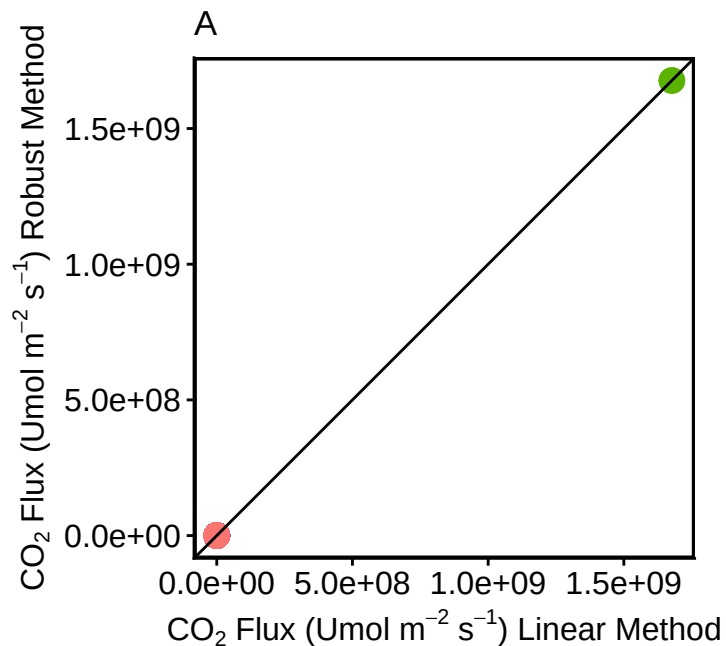
```

allflux_filtered <- fluxes_flagged %>% filter(!CO2_flux_rejected)

p1 <- ggplot(allflux_filtered, aes(lin_flux.estimate, rob_flux.estimate, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  labs(title = "A") +
  ylab(expression(paste(CO2, " Flux (Umol m-2 s-1) Robust Method"))) +
  xlab(expression(paste(CO2, " Flux (Umol m-2 s-1) Linear Method"))) +
  guides(color = guide_legend(title = "Plot")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)

p1

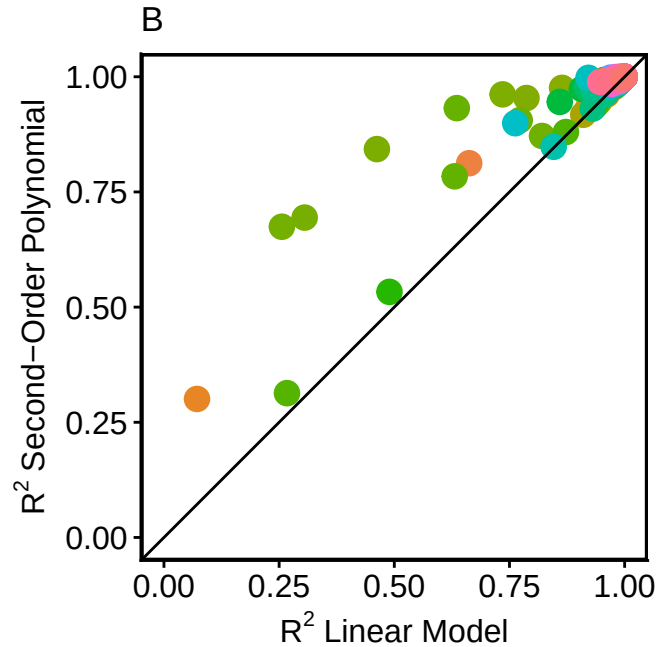
```



```
p2 <- ggplot(allflux_filtered, aes(lin_r.squared, poly_r.squared, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  xlim(0,1) + ylim(0,1) +
  labs(title = "B") +
  ylab(expression(paste("R"2* " Second-Order Polynomial")))) +
  xlab(expression(paste("R"2* " Linear Model")))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
```

p2

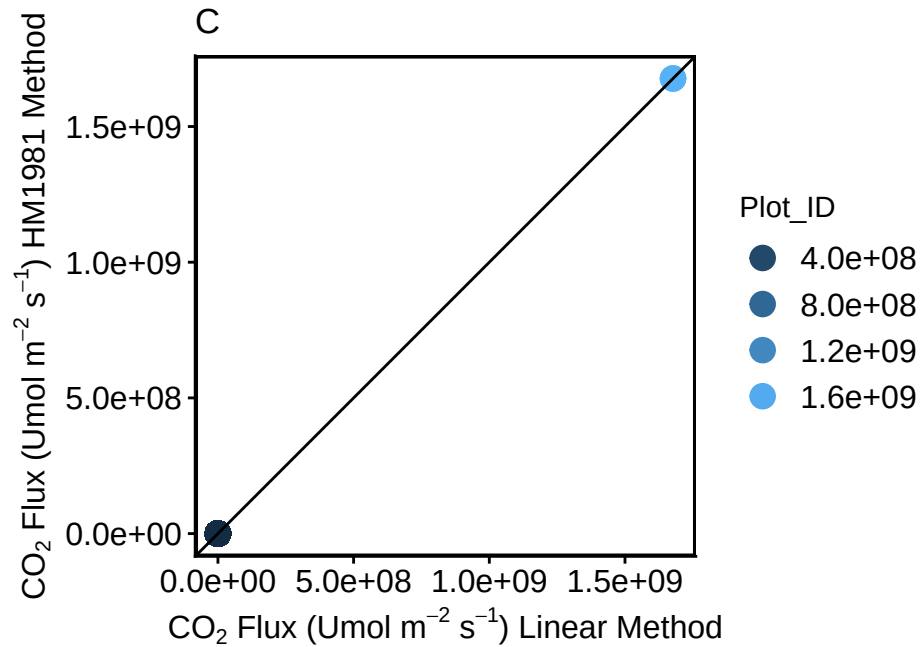
Warning: Removed 2 rows containing missing values or values outside the scale range (`geom\_point()`).



```
p3 <- ggplot(allflux_filtered, aes(lin_flux.estimate, HM81_flux.estimate, color = lin_r.squa
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  #xlim(0,50) + #ylim(0,50) +
  labs(title = "C") +
  ylab(expression(paste( CO [2], " Flux (Umol m-2* " s-1*) HM1981 Method"))) +
  xlab(expression(paste( CO [2], " Flux (Umol m-2* " s-1*) Linear Method"))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="right") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
```

p3

Warning: Removed 67 rows containing missing values or values outside the scale range (``geom_point()``).



```
#ggarrange(p1, p2, p3, ncol=3, nrow=1, common.legend = TRUE, legend="none")
```

21 Read out final dataframe and export for use elsewhere

```
allflux_filtered_renamed <- fluxes_flagged %>%
  mutate(
    Date = mdy(str_extract(Plot, "[0-9]+/[0-9]+/[0-9]{4}")),
    Month = month.abb[month(Date)],
    Site = str_extract(Plot, "(PTR|OWC|CRC)"),
    Zone = str_extract(Plot, "_TR_|_UP_") %>%
      str_remove_all("_"),
    Replicate = as.integer(
      str_extract(Plot, "(?<=_ (TR|UP)_)[0-9]+")
    ),
    Row_ID = as.integer(
      str_extract(Plot, "[0-9]+$") ) %>%

# Optional: rebuild a clean Plot label
  mutate(
    Plot_clean = paste(
```

```

        format(Date, "%Y-%m-%d"),
        Site,
        Zone,
        Replicate,
        Row_ID,
        sep = "_"
    )
) %>%

select(
  Plot_clean,
  Date,
  Month,
  Site,
  Zone,
  Replicate,
  Row_ID,
  everything(),
  -Plot
)

#export as a csv to the folder that you want
write.csv(allflux_filtered_renamed, file="Processed Data/COMPASS_Synoptic_LE_TreeStem_Fluxes.csv")
# #Change file name

```

22